ML Engineering for Production (DSAI 406)

Lecture 5

Mohamed Ghalwash
mghalwash@zewailcity.edu.eg

Recording is NOT allowed

Recap

- CI/CD
- Githib Actions
- Anatomy of YAML

```yaml
name: ...
on: ...

jobs:
  ID:
    runs-on: ...
    steps:
      - name: ...
        uses/run: ...
        with:
          param: ...
```

The CI/CD Workflow: End-to-End MLOps Pipeline

```yaml
name: MLOps End-to-End Pipeline

on:
  push:
    branches: [ dev ]
  pull_request:
    branches: [ main ]

jobs:
  # STEP 1: Continuous Integration (The "Is the code safe?" part)
  test_and_validate:
    runs-on: ubuntu-latest
    steps:
        ...
  # STEP 2: Continuous Delivery (The "Package and Ship" part)
  build_and_deploy:
    needs: test_and_validate # why?
    runs-on: ubuntu-latest
    steps:
      ...
```

Automated Testing for ML

In MLOps, we move beyond simple syntax checks. We need **three levels** of testing to ensure both the code and the model are healthy:

Test Type	What it checks	Example
Unit Tests	Individual code components	`test_preprocess_nulls()` : Does the scaler handle NaNs?
Integration Tests	Data + Model pipeline	`test_input_shape()` : Does the model accept the current data?
Validation Tests	Model "Behavioral" Performance	`assert accuracy > 0.80` : Is the model good enough to deploy?

Validation Testing

```python
# tests/test_model_logic.py

def test_prediction_range():
    model = load_model("models/latest.pkl") # hmmmm?
    prediction = model.predict(test_sample)
    # Ensure probabilities are always between 0 and 1
    assert prediction >= 0 and prediction <= 1

def test_overfitting_check():
    # A simple test to ensure loss isn't exactly zero
    assert metrics['train_loss'] > 1e-5
```

From Experiment to Production

Once the PR is approved, the **CD Pipeline** takes over. But where are the pieces?

1. The Blueprint

The **Dockerfile** is in your Git repo. It defines the OS, Python version, and system dependencies.

2. The Brain

The **Model Weights** are pulled from **MLflow**. We use a script to find the `Best Model` or `Production` tag.

3. The Container

Docker builds the image, downloads the model inside it, and starts the API service.

Use Cases

Step 1: Simple Trigger

The Goal: Automatically train the model when code changes

Step 1: Simple Trigger

The Goal: Automatically train the model when code changes

```yaml
name: Simple CI
on: [push]

jobs:
  train-model:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.10'
      - run: pip install -r requirements.txt
      - run: python train.py
```

Step 1: Simple Trigger

The Goal: Automatically train the model when code changes

```yaml
name: Simple CI
on: [push]

jobs:
  train-model:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.10'
      - run: pip install -r requirements.txt
      - run: python train.py
```

Issue

This is "blind" training. It runs the code, but we don't know if the model is actually better than the previous one, and we aren't saving the results anywhere

Step 2: Adding Observability

The Goal: Log results using MLFlow

Step 2: Adding Observability

The Goal: Log results using MLFlow

```yaml
# ...
# (same as previous)
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.10'
      - run: pip install -r requirements.txt mlflow
      - name: Train and Log
        run: python train.py --epochs 1 # Validation Test
```

Step 2: Adding Observability

The Goal: Log results using MLFlow

```yaml
# ...
# (same as previous)
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.10'
      - run: pip install -r requirements.txt mlflow
      - name: Train and Log
        env:
          MLFLOW_TRACKING_URI: ${{ secrets.MLFLOW_TRACKING_URI }}
        run: python train.py
```

Step 2: Adding Observability

The Goal: Log results using MLFlow

```yaml
# ...
# (same as previous)
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.10'
      - run: pip install -r requirements.txt mlflow
      - name: Train and Log
        env:
          MLFLOW_TRACKING_URI: ${{ secrets.MLFLOW_TRACKING_URI }}
        run: python train.py
```

Issue

Great, we are logging! But we are just "blindly" pushing code. What if the model in this push is worse than the one currently in production? We need to fetch the best model to compare or deploy

Step 3: Fetching the Best Model

The Goal: Finds the best model

Step 3: Fetching the Best Model

The Goal: Finds the best model

get_best_model.py

```python
# Finds the best model and saves its URI to a file
import mlflow

client = mlflow.tracking.MlflowClient()
exp = client.get_experiment_by_name('...')
runs = client.search_runs(
exp.experiment_id,
order_by=['metrics.accuracy DESC'],
max_results=1)
with open('best_model_uri.txt', 'w') as f:
    f.write(runs[0].info.artifact_uri + '/model')
```

Step 3: Fetching the Best Model

The Goal: Finds the best model

```yaml
# ...
  steps:
    # ... (install steps)
    - name: Fetch Best Model
      env:
        MLFLOW_TRACKING_URI: ${{ secrets.MLFLOW_TRACKING_URI }}
      run: python get_best_model.py
```

get_best_model.py

```python
# Finds the best model and saves its URI to a file
import mlflow

client = mlflow.tracking.MlflowClient()
exp = client.get_experiment_by_name('...')
runs = client.search_runs(
exp.experiment_id,
order_by=['metrics.accuracy DESC'],
max_results=1)
with open('best_model_uri.txt', 'w') as f:
    f.write(runs[0].info.artifact_uri + '/model')
```

Step 3: Fetching the Best Model

The Goal: Finds the best model

```yaml
# ...
  steps:
    # ... (install steps)
    - name: Fetch Best Model
      env:
        MLFLOW_TRACKING_URI: ${{ secrets.MLFLOW_TRACKING_URI }}
      run: python get_best_model.py
```

Issue
This YAML validates the model but does not build the docker image

get_best_model.py

```python
# Finds the best model and saves its URI to a file
import mlflow

client = mlflow.tracking.MlflowClient()
exp = client.get_experiment_by_name('...')
runs = client.search_runs(
exp.experiment_id,
order_by=['metrics.accuracy DESC'],
max_results=1)
with open('best_model_uri.txt', 'w') as f:
    f.write(runs[0].info.artifact_uri + '/model')
```

Step 4: Building Docker

Step 4: Building Docker

```yaml
# ...
  steps:
    # ... (as previous steps)
    - name: Docker Login
      uses: docker/login-action@v3
      with:
        username: ${{ secrets.DOCKER_USERNAME }}
        password: ${{ secrets.DOCKER_PASSWORD }}

    - name: Build and Push Docker Image
      run: |
        MODEL_URI=$(cat best_model_uri.txt)
        docker build -t ${{ secrets.DOCKER_USERNAME }}/ml-app:latest .
        docker push ${{ secrets.DOCKER_USERNAME }}/ml-app:latest
```

Step 4: Building Docker

```yaml
# ...
  steps:
    # ... (as previous steps)
    - name: Docker Login
      uses: docker/login-action@v3
      with:
        username: ${{ secrets.DOCKER_USERNAME }}
        password: ${{ secrets.DOCKER_PASSWORD }}

    - name: Build and Push Docker Image
      run: |
        MODEL_URI=$(cat best_model_uri.txt)
        docker build --build-arg MODEL_PATH=$MODEL_URI -t ${{ secrets.DOCKER_USERNAME }}/ml-app:latest .
        docker push ${{ secrets.DOCKER_USERNAME }}/ml-app:latest
```

Step 4: Building Docker

```yaml
# ...
  steps:
    # ... (as previous steps)
    - name: Docker Login
      uses: docker/login-action@v3
      with:
        username: ${{ secrets.DOCKER_USERNAME }}
        password: ${{ secrets.DOCKER_PASSWORD }}

    - name: Build and Push Docker Image
      run: |
        MODEL_URI=$(cat best_model_uri.txt)
        docker build --build-arg MODEL_PATH=$MODEL_URI -t ${{ secrets.DOCKER_USERNAME }}/ml-app:latest .
        docker push ${{ secrets.DOCKER_USERNAME }}/ml-app:latest
```

Issue

This YAML is getting huge. We need to separate Validation from Deployment.

Step 5: Separation of Concerns (Multi-Job)

Step 5: Separation of Concerns (Multi-Job)

```yaml
jobs:
  validate: # CI
    runs-on: ubuntu-latest
    steps:
      - run: python train.py --smoke-test

  deploy: # CD
    runs-on: ubuntu-latest
    steps:
      - run: docker build ...
```

Step 5: Separation of Concerns (Multi-Job)

```yaml
jobs:
  validate: # CI
    runs-on: ubuntu-latest
    steps:
      - run: python train.py --smoke-test

  deploy: # CD
    runs-on: ubuntu-latest
    steps:
      - run: docker build ...
```

Issue

Parallelism! GitHub runs jobs in parallel by default. The deploy job will start before the validate job finishes. We might deploy a broken model!

Step 6: The Final Pipeline (Dependencies)

```yaml
on:
  push:
    branches: [main]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - name: Dry Run & Pytest
        run: pytest tests/ && python train.py --epochs 1

  deploy:
    needs: validate # <--- THE FIX: Wait for validation to pass 🟢
    runs-on: ubuntu-latest
    steps:
      - name: Fetch the best model
        run: python get_best_model.py
      - name: Build docker
        run: |
          docker build --build-arg MODEL_PATH=$(cat best_model_uri.txt) -t app:latest .
          docker push app:latest
```

Docker

```
# 1. Start with a lightweight Python base
FROM python:3.10-slim

# 2. Define the Build Argument. This will be passed from your GitHub Action
ARG MODEL_PATH

# 3. Set environment variables
ENV MODEL_DIR=/opt/ml/model

# 4. Install system dependencies & libraries
COPY requirements.txt .
RUN apt-get update && apt-get install -y --no-install-recommends build-essential
RUN pip install --no-cache-dir -r requirements.txt

# 5. Fetch the Model from MLflow during Build
RUN mkdir -p ${MODEL_DIR}
RUN mlflow artifacts download --artifact-uri ${MODEL_PATH} --dst-path ${MODEL_DIR}

# 6. Copy your code
WORKDIR /app
COPY src/ ./
```

Data Version Control (DVC)

Git for Data

Data: Why Git is Fundamentally Broken for Data

Git was built for **source code**, not **big data**

- **Binary Blobs:** Git is designed for text line-diffs. A `.parquet` or `.h5` file is a "black box" to Git.
- **Repo Bloat:** Every time you change a 100MB dataset, Git stores a *full new copy* in the `.git` folder.
- **The Link Problem:** How do you prove which specific version of the data created `model_v1.pt`?

DevOps

Code Version A ➔ Binary A

MLOps

Code A + Data B ➔ Model C

How DVC Works

- DVC keeps the **Heavy Data** in external storage and keeps a **Lightweight Pointer** in Git

The Command

```
# Track the file
dvc add data/raw.csv

# Link to Git
git add data/raw.csv.dvc
git commit -m "Add raw data"
```

The Pointer (.dvc)

```
# Actual content of raw.csv.dvc
outs:
- md5: a1b2c3d4e5f6g7h8...
  size: 1073741824
  path: raw.csv
```

How DVC Works

- DVC keeps the **Heavy Data** in external storage and keeps a **Lightweight Pointer** in Git

The Command

```
# Track the file
dvc add data/raw.csv

# Link to Git
git add data/raw.csv.dvc
git commit -m "Add raw data"
```

The Pointer (.dvc)

```
# Actual content of raw.csv.dvc
outs:
- md5: a1b2c3d4e5f6g7h8...
  size: 1073741824
  path: raw.csv
```

When a teammate runs git pull, they get the Pointer. When they run `dvc pull`, DVC fetches the exact 1GB file matching that MD5 hash.

Connecting the Dots: Docker + MLflow + CI

Every tool we have learned so far—**Docker**, **DVC**, and **MLflow**—now converges into a single automated pipeline. This is where MLOps becomes a "system" rather than just a collection of scripts.

- 🐳 **Docker:** The CI Runner uses your **Dockerfile** to ensure the test environment perfectly matches your development environment. No "it worked on my machine."
- 📈 **MLflow:** The CI process logs the results of the "Test Run" to your tracking server. You can compare CI performance against your local manual experiments.
- 🗃️ **DVC:** The CI pipeline pulls the specific version of the data from remote storage to run validation tests.

The Pipeline Workflow:
`Git Push` ➔ `Pull Data (DVC)` ➔ `Run Tests (Pytest)` ➔ `Log to MLflow` ➔ `Build Container` ➔ `Deployment Approval`

Learn More

Course Homepage